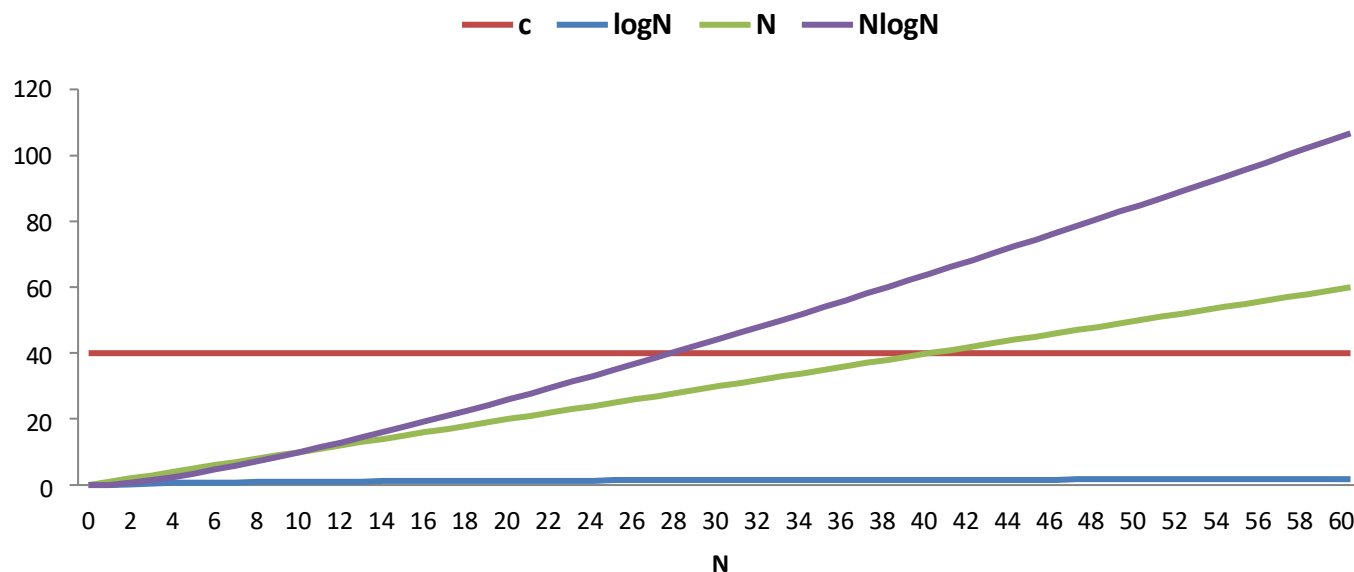


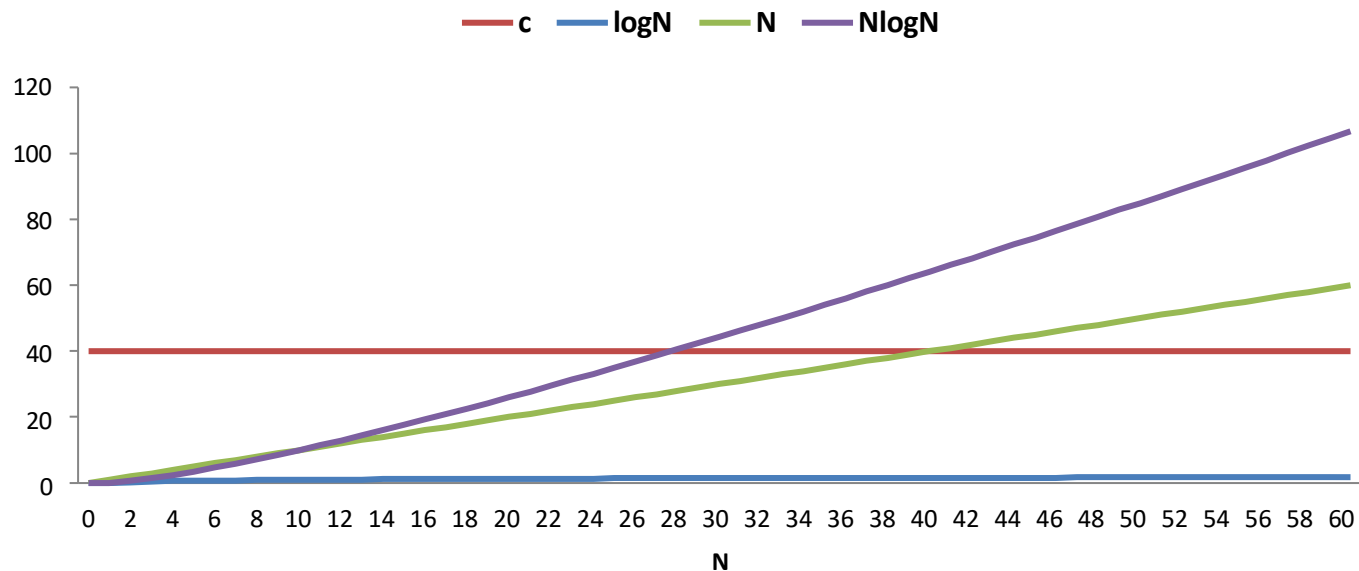
Estrutura de Dados I

Notações Assintóticas

- Motivação: Definir ordem entre funções.
- Avaliação pontual $\rightarrow$ Não tem sentido:
 - $f(N) < g(N)$?



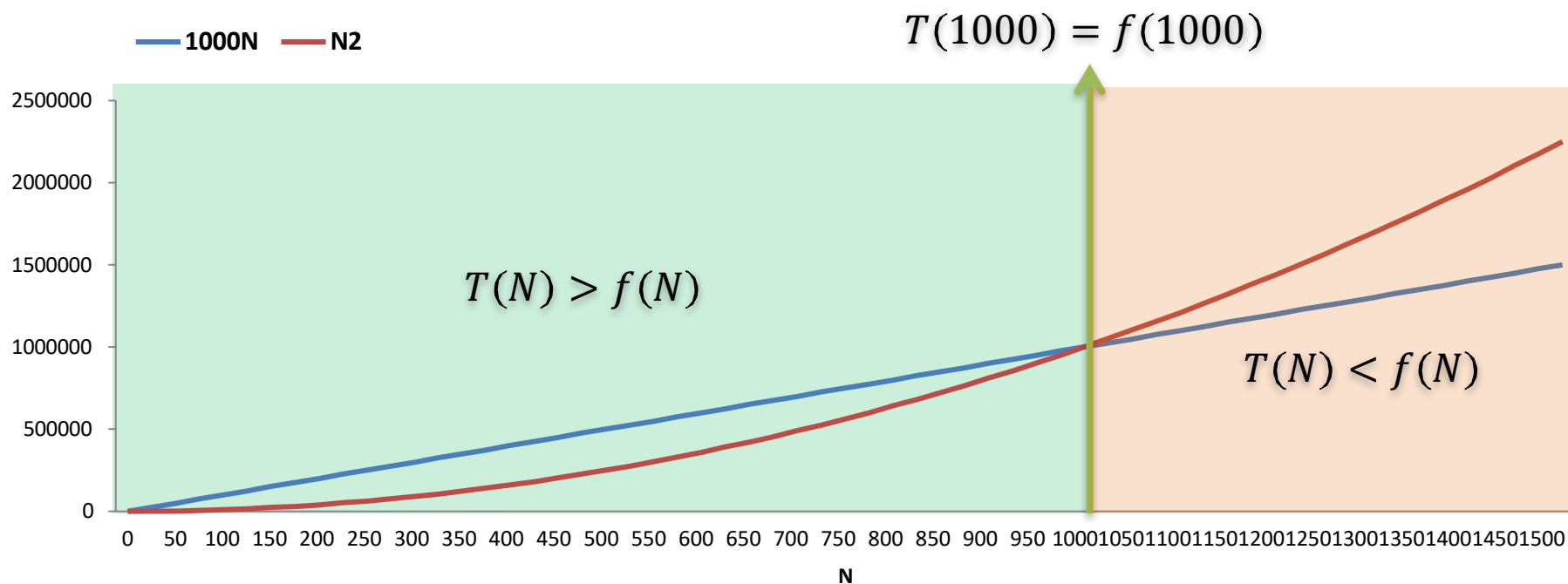
Cor	Função	Nome	Tipo de Crescimento	Comportamento
Vermelho	c	Constante	Constante	Fica fixa (melhor possível)
Azul	$\log N$	Logarítmico	Muito lento	Cresce bem devagar
Verde	N	Linear	Linear	Cresce em linha reta
Roxo	$N \log N$	Linearítmico	Mais rápido que linear	Cresce mais rápido que N



- Forma de Análise: Taxa de crescimento.

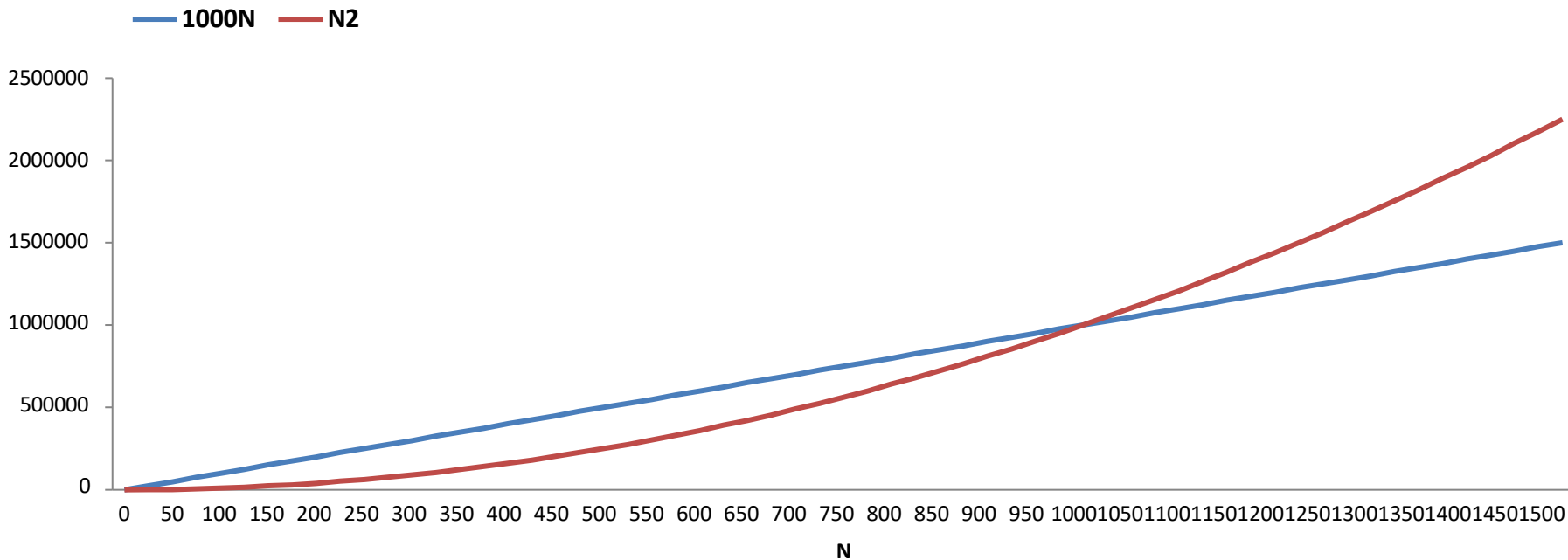
$$T(N) = 1000N$$

$$f(N) = N^2$$



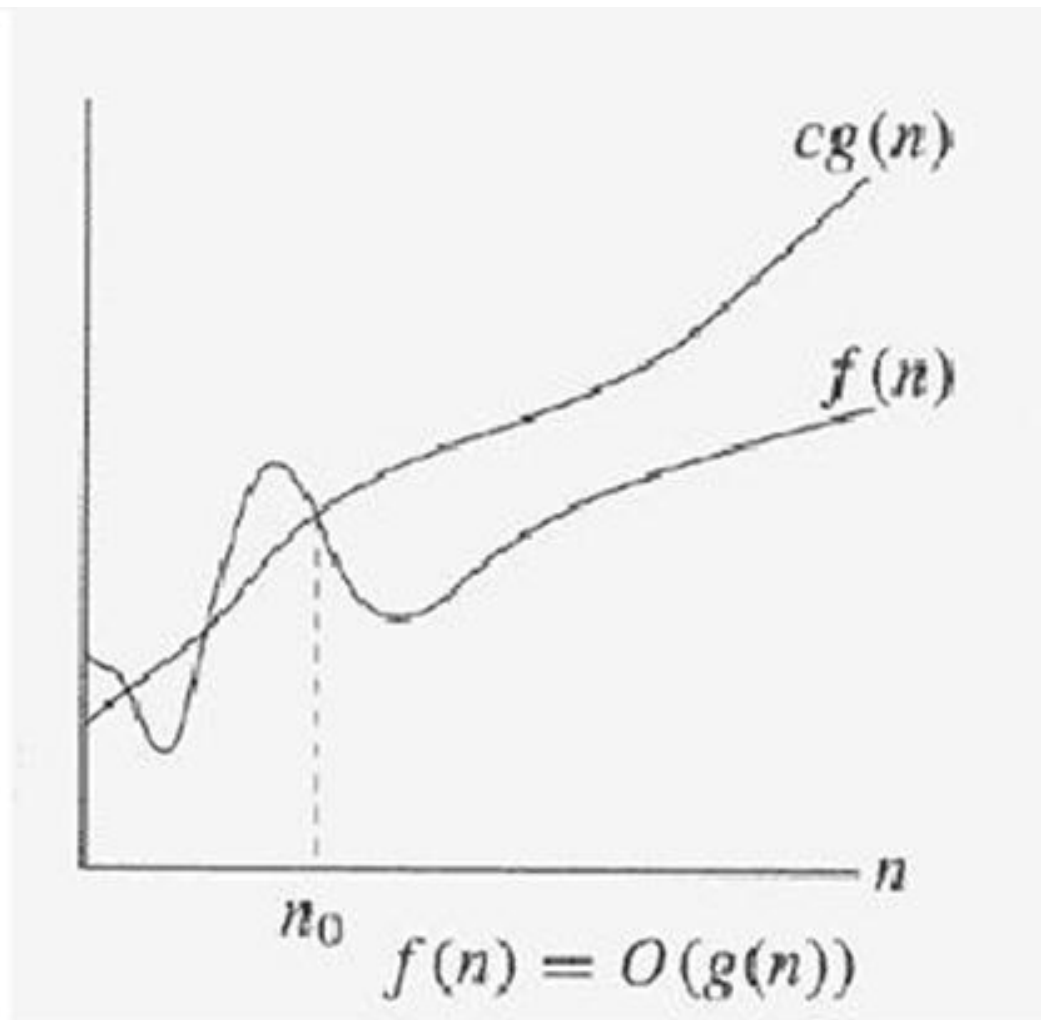
- Apesar de $1000N$ ser maior que N^2 para N pequenos, a taxa de crescimento de N^2 é maior e ultrapassa $1000N$ para $N > 1000$. Logo, N^2 é maior que $1000N$.
- Há um valor de N (n_0) a partir do qual $c.f(N)$ será sempre, no mínimo, tão grande quanto $T(N)$. Neste caso: $n_0=1000$ e $c=1$. Outra possibilidade seria $n_0=100$ e $c=10$.

- Concluindo, podemos dizer que a taxa de crescimento de $T(N)=1000N$ é menor ou igual à taxa de crescimento de $f(N)=N^2$.



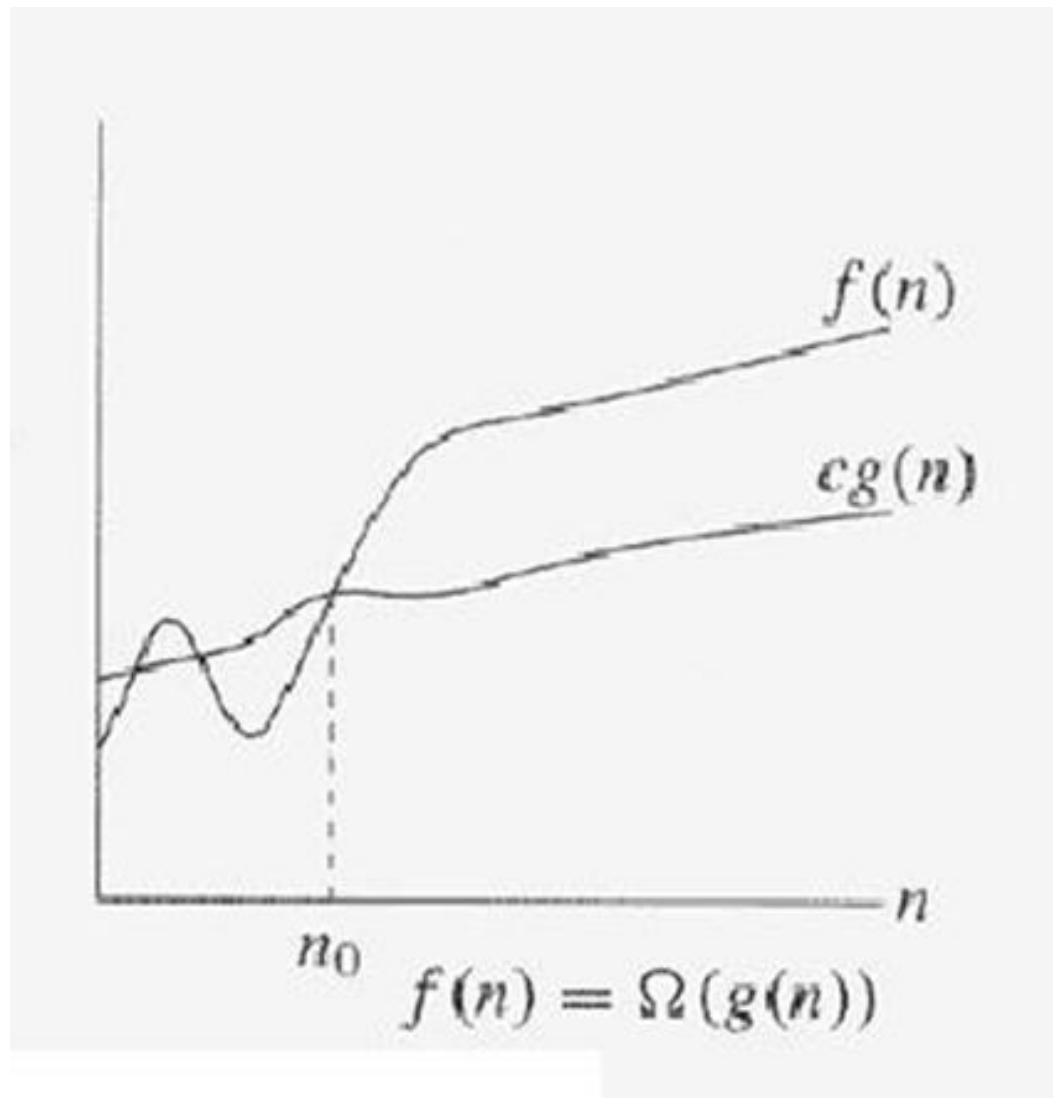
$T(N) = O(f(N))$, se houver as constantes positivas c e n_o , tal que $T(N) \leq cf(N)$, quando $N \geq n_o$.

- **Se $T(N) = O(f(N))$:**
 - Estamos **garantindo** que a função $T(N)$ **cresce no máximo** na mesma taxa que $f(N)$.
 - Ou seja, $T(N)$ **não cresce mais rápido** que $f(N)$ (para N grande).
- **$f(N)$ é o limite superior de $T(N)$:**
 - $f(N)$ serve como um **teto** (upper bound) para o comportamento de $T(N)$.
 - Pode ser um teto “folgado” (não apertado), mas é válido.



$T(N) = \Omega(f(N))$, se houver as constantes positivas c e n_o , tal que $T(N) \geq cf(N)$, quando $N \geq n_o$.

- **Se $T(N) = \Omega(g(N))$:**
 - Estamos **garantindo** que a função $T(N)$ **cresce no mínimo** na mesma taxa que $g(N)$.
 - Ou seja, $T(N)$ **não cresce mais devagar** que $g(N)$ (para N grande).
- **$g(N)$ é o limite inferior de $T(N)$:**
 - $g(N)$ serve como um **piso** (lower bound) para o comportamento de $T(N)$.
 - Significa que o algoritmo **não pode ser melhor** que essa ordem de crescimento (em termos assintóticos).



$T(N) = \Theta(h(N))$, se e somente se $T(N) = O(h(N))$ e $T(N) = \Omega(h(N))$.

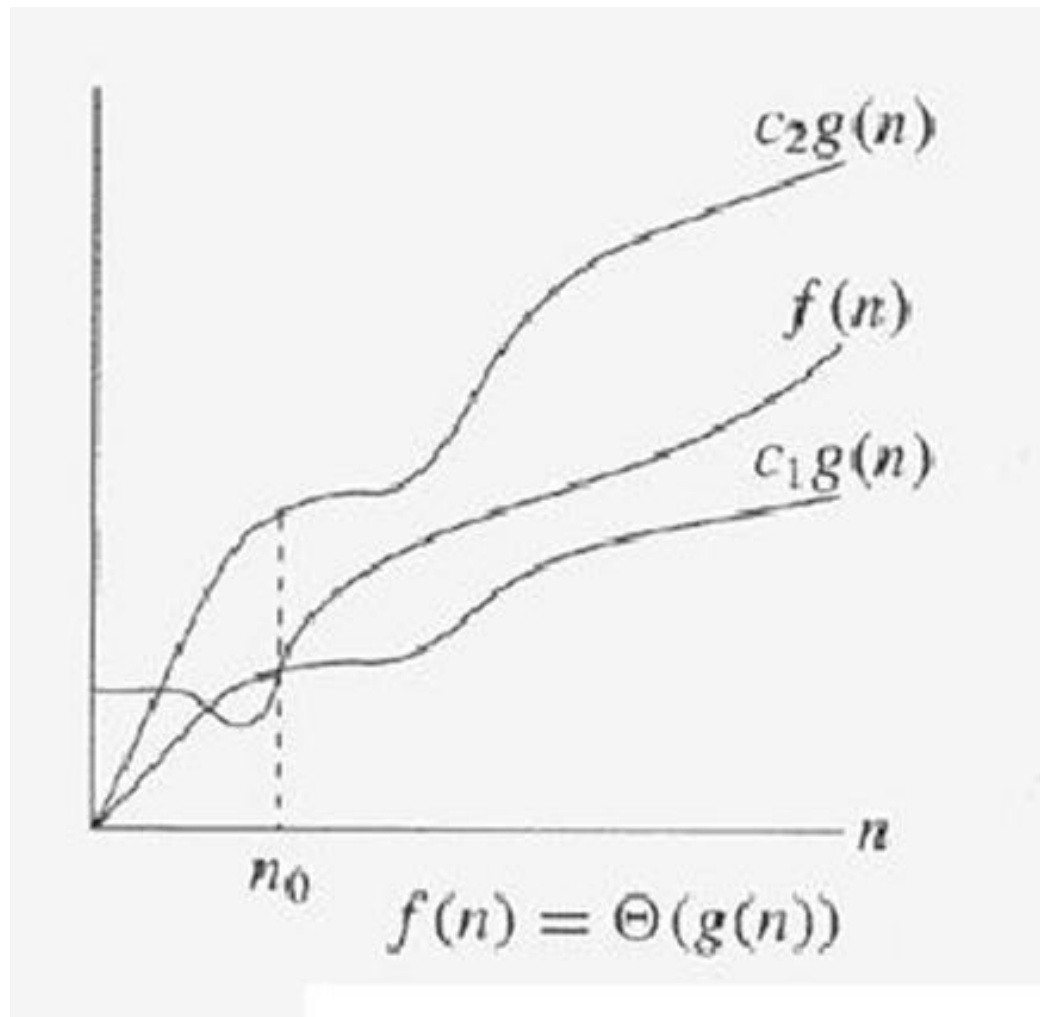
- Se $T(N) = \Theta(h(N))$: estamos garantindo que a função $T(N)$ cresce a uma taxa igual à $h(N)$.

O que isso realmente significa?

Quando dizemos $T(N) = \Theta(h(N))$, estamos garantindo que a função $T(N)$ cresce **exatamente** na mesma taxa que $h(N)$.

Não é “no máximo” (como O) nem “no mínimo” (como Ω).

É “**nem mais, nem menos**” — o comportamento é **assintoticamente equivalente**.



Notação	Tipo de Limite	Desigualdade	Significado prático	Exemplo comum
O	Superior (Upper)	$T(N) \leq c \cdot h(N)$	“No máximo”	Busca Sequencial = $O(n)$
Ω	Inferior (Lower)	$T(N) \geq c \cdot h(N)$	“No mínimo”	Busca Sequencial = $\Omega(1)$
Θ	Apertado (Tight)	$c_1 \cdot h(N) \leq T(N) \leq c_2 \cdot h(N)$	Exatamente da mesma ordem	Busca Sequencial = $\Theta(n)$

Algoritmo	Complexidade Θ	Significado prático
Busca Sequencial	$\Theta(n)$	Sempre linear no caso médio/pior
Busca Binária	$\Theta(\log n)$	Sempre logarítmica
Merge Sort	$\Theta(n \log n)$	Sempre $n \log n$ (melhor e pior caso)
Insertion Sort	$\Theta(n^2)$	Quadrático no pior caso e médio
Quick Sort (médio)	$\Theta(n \log n)$	Muito usado na prática

$$N^2 = O(N^3)$$

$$N^3 = \Omega(N^2)$$

$$f(N) = N^2, \quad g(N) = 2N^2 + N$$

$$f(N) = O(g(N)) \text{ e } f(N) = \Omega(g(N)) \rightarrow f(N) = \Theta(g(N))$$

$$f(N) = 2N^2 + 3N$$

$$f(N) = O(N^4), f(N) = O(N^3), \mathbf{f(N) = \Theta(N^2)}$$

Colocando:

$$\mathbf{f(N) = \Theta(N^2)}$$

Implica não só que $f(N) = O(N^2)$, mas também que N^2 é a ordem de crescimento que melhor se ajusta ao comportamento assintótico de $f(N)$

- Se $T_1(N) = O(f(N))$ e $T_2(N) = O(g(N))$, então:
 - a) $T_1(N) + T_2(N) = O(f(N) + g(N))$;
 - b) $T_1(N) \times T_2(N) = O(f(N) \times g(N))$;
- Se $T(N)$ é um polinômio de ordem k , então:
 - a) $T(N) = \Theta(N^k)$
- $\log^k N = O(N)$ para qualquer constante k . Isto indica que logaritmos crescem muito lentamente.

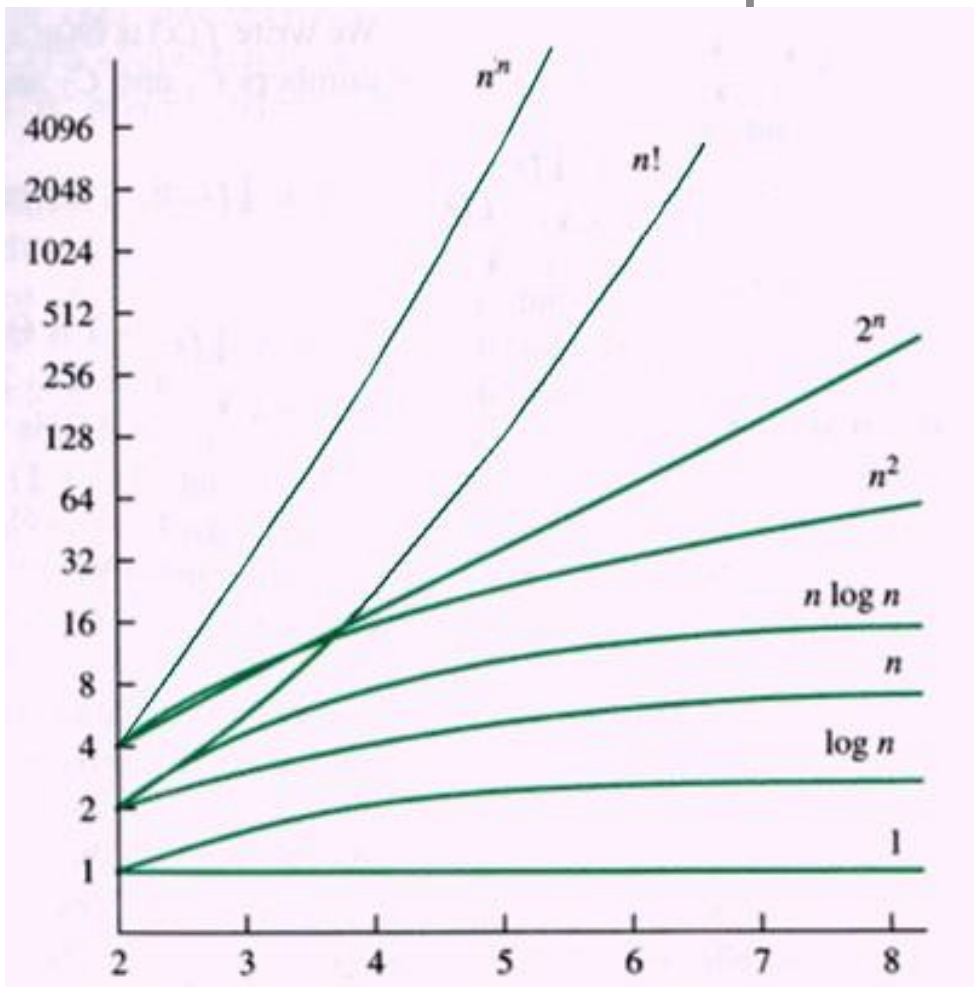
Função	Ordem de crescimento	Velocidade relativa
$(\log N)^{100}$	Logarítmico	Extremamente lento
N	Linear	Muito mais rápido
N^2	Quadrático	Ainda mais rápido

Propriedade	Resultado	Utilidade
Soma	$O(f + g)$	Análise de algoritmos que fazem várias etapas
Multiplicação	$O(f \times g)$	Análise de loops aninhados
Polinômio de grau k	$\Theta(N^k)$	Simplifica análise de equações
Logaritmos	$\log^k N = O(N)$	Mostra que log é "quase constante"

- Taxas de Crescimento Típicas:

Taxa de Crescimento	Nome da Ordem	Nome em Português	Comportamento	Exemplo de Algoritmo
c	Constante	Constante	Não depende do tamanho da entrada	Acesso a um array por índice
$\log N$	Logarítmica	Logarítmica	Cresce muito devagar	Busca Binária
$\log^2 N$	Logarítmica quadrática	Log quadrática	Ainda cresce bem devagar	Algoritmos avançados (raros)
N	Linear	Linear	Crescimento proporcional ao tamanho	Busca Sequencial
$N \log N$	Linearítmica	Linearítmica	Um pouco pior que linear	Merge Sort , Quick Sort (médio)
N^2	Quadrática	Quadrática	Aceitável até ~10.000 elementos	Bubble Sort, Insertion Sort
N^3	Cúbica	Cúbica	Aceitável até ~few hundred elements	Algoritmos de multiplicação ingênua
2^N	Exponencial	Exponencial	Explosivo — impraticável para N grande	Subconjuntos (Power Set), Backtracking

- Taxas de Crescimento Típicas:



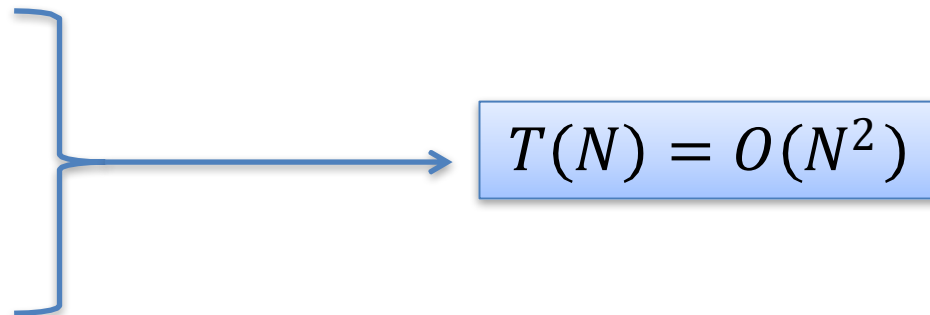
- Considere quaisquer funções com, por exemplo, uma variável independente n (como $n+10$ ou n^2+1):
- A análise de algoritmos ignora os valores pequenos e concentra-se em instâncias grandes para n ($n \rightarrow +\infty$).
- Para valores enormes de n , as funções n^2 , $(3/2)n^2$, $9999n^2$, $n^2/1000$, n^2+100n crescem todas com a mesma velocidade.
- Esse tipo de análise matemática é chamada *assintótica*. Nessa análise, as funções são classificadas em ordens.
- As cinco funções acima, por exemplo, pertencem à mesma ordem e são, portanto, equivalentes.

- Assim, podemos não considerar constantes ou termos de ordens baixas:

a) $T(N) = O(2N^2)$,

b) $T(N) = O(N^2 + N)$,

c) $T(N) = O(N^2 + 7)$,


$$T(N) = O(N^2)$$

- Não faz sentido:

a) $f(N) \leq O(g(N))$,

b) $f(N) \geq O(g(N))$,